Java Access modifiers

Interview Notes

What is access modifiers in Java ?

Who can access the class members (variable, method, constructor, classes)

Purpose:

- → Data hiding
- → Encapsulation
- → security
- → Controlled Interaction between classes

In real Example

data members are usually Private and It is accessed through method like (getter/setter)

Java Access Modifier

4 type	Keyword
Public	Public
Protected	Protected
Default	no modifier
Private	Private

Access	class	Same Package	sub class same pkg	sub class different pkg	World
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
default	✓	✓	✓	✗	✗
Private	✓	✗	✗	✗	✗

✓ = Accessible

✗ = Not accessible

Public

Accessible Every where

```
Public class A {
    public int x;
}
```

Uses:

→ It is used for API and the services

Private :

Accessible only Inside the Package

Ex:

```
class A{
    private int x;
}
```

Note :x.

→ Private members are Inherited

→ It means It is not directly accessible because they belong to the declaring class

→ It mean the member be come Part of child object

Ex:

A correct way to access private member

```
class Parent {
    private int money = 1000;
    Public int getx(){
        return money;
    }
}
```

```
class child extends Parent {
    void show() {
        System.out.Println (getx())
    }
}
```

→ Always memory exist but direct access is not allowed

→ O/Ps accessible in same Package

Protected

→ same Package

→ different Package but in that only sub class

→ up casting allowded

→ Accessing Protected via Parent reference → Not allowded (outside Package)

Where use?

Private → variables

Public → Method

Protected / Private → Helpers

It is for mainly Encapsulation

singleton class

→ It is can be accessed only one time

→ If we try to access more than one's we get the obj 1 's reference variable address

Code

```
Public class Singleton {

    Private static Singleton instance;
    Public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
```